Extending WSO2 API Manager



CC by 4.0

WSO2

Mediation Extensions

WSO2

Mediation Extensions - API Policies

- Mediation flows intercept and modify messages that are passed between services and clients that want to use those services
- The API Gateway has a default mediation flow, which you can extend by adding custom policies to API operations.
- There is a set of API policies by default shipped with the product, so that users can use and they can extend that list by adding more custom policies, either API level or common to all APIs.

API Policies Overview

Mediation Extensions - API Policies

- Mediation flows intercept and execute a set of rules based on the message that are passed between services and clients that want to use those services.
- The API Gateway has a default mediation flow, which you can extend by adding custom policies.
- In API Manager there are 3 default sequences engaged as in (request flow), out(response flow) and fault (fault flow).
- You can create a custom API policy either manually or using WSO2 tooling support (i.e., WSO2 Developer Studio), and then engage it per operation of API.
- With this you can modify the default mediation flow for different usabilities according to your requirement. i.e.

    Log the request/response message

    Execute operations on message context properties

    Customize or format the requests and responses

Custom Handlers

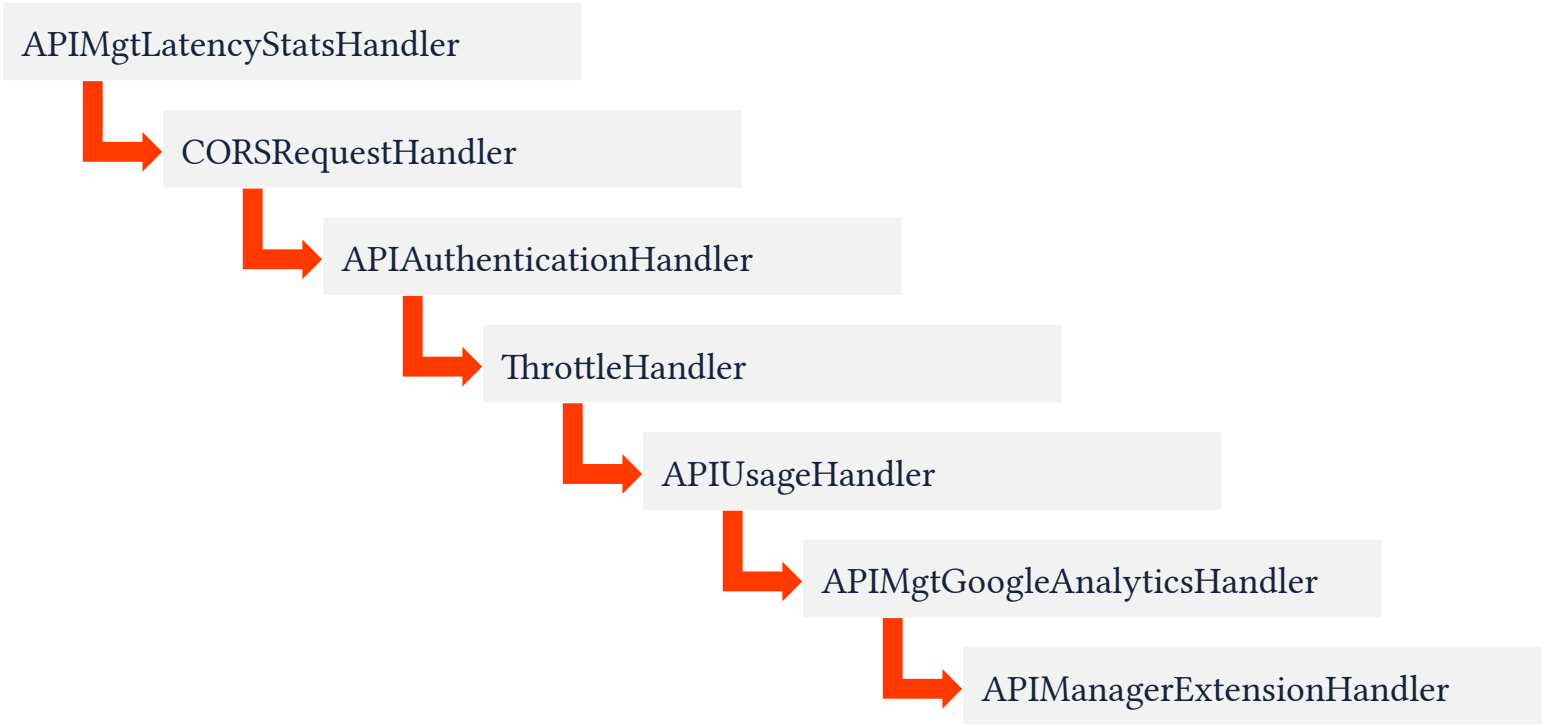


Handlers

- When an API is created, a file with its synapse configuration is deployed to the memory API Gateway.
- To retrieve the deployed synapse configuration, product REST API “WSO2 Gateway API v2” can be used. [Refer]
- It has a set of handlers, each of which is executed on the APIs in the same order they appear in the configuration.
- You find the default handlers in any API's Synapse definition as shown below.
- API Manager is extensible by writing custom handlers.

```
<handlers>
      <handler class="org.wso2.carbon.apimgt.gateway.handlers.common.APIMgtLatencyStatsHandler"/>
      <handler class="org.wso2.carbon.apimgt.gateway.handlers.security.CORSRequestHandler">
         <property name="apiImplementationType" value="ENDPOINT"/>
      </handler>
      <handler class="org.wso2.carbon.apimgt.gateway.handlers.security.APIAuthenticationHandler"/>
      <handler class="org.wso2.carbon.apimgt.gateway.handlers.throttling.ThrottleHandler"/>
      <handler class="org.wso2.carbon.apimgt.usage.publisher.APIMgtUsageHandler"/>
      <handler class="org.wso2.carbon.apimgt.usage.publisher.APIMgtGoogleAnalyticsTrackingHandler">
         <property name="configKey" value="gov:/apimgt/statistics/ga-config.xml"/>
      </handler>
      <handler class="org.wso2.carbon.apimgt.gateway.handlers.ext.APIManagerExtensionHandler"/>
   </handlers>
```

Handler Processing Order



Handler	Usage
APIMgtLatencyStatsHandler	To set parameters that are needed to calculate API latency-related statistics.
CORSRequestHandler	Sets the CORS headers to the request and executes the CORS sequence mediation logic. This handler is thereby responsible for returning the CORS headers from the gateway or routing the requests to the backend and letting the backend send the CORS headers.
APIAuthenticationHandler	Validates the token/ssl flows used to invoke the API. It also determines whether the token is of type Production or Sandbox and sets MessageContext variables appropriately.
APIThrottleHandler	Rate limits requests based on the rate limiting policy specified by the policyKey property. Rate limiting is applied both at the application level as well as subscription level.

Handler	Usage
APIMgtUsageHandler	Publishes events to WSO2 API Manager Analytics server for collection and analysis of statistics. This handler only comes to effect if API usage tracking is enabled.
APIMgtGoogleAnalyticsTrackingHandler	Publishes events to Google Analytics. This handler only comes into effect if Google analytics tracking is enabled.
APIManagerExtensionHandler	Extends the mediation flow of messages passing through the API Gateway.

Custom Handlers

- You can engage a custom handler to all APIs at once or only to selected APIs
- To engage to all APIs, the recommended approach is to add it to the *<APIM_HOME>/repository/resources/api_templates/velocity_template.xml* file.
- To engage handlers to a single API you need to edit the velocity template so that the handler will be applied to the selected API, based on a condition.
- Once you change the velocity template, you need to deploy a new revision of the API.

Writing a Custom Handler

Workflow Extensions

WSO2

Workflow Extensions

- Provides an extension point to engage custom workflow.
- Extensions are possible for a number of processes and flows
- Extensions could also be written to redirect to an HTTP URL.
- Workflow extensions can be used to attach a workflow to the following Developer Portal/API

  Publisher operations:

  Adding an Application Creation Workflow
  Adding an Application Key Generation Workflow
  Adding an API Subscription Workflow
  Adding an API Subscription Deletion Workflow
  Adding an API Subscription Tier Update Workflow
  Adding a User Signup Workflow
  Adding an API State Change Workflow
  Adding a Revision Deployment Workflow

Steps to Customize a Workflow Extension

- Inherited from the org.wso2.carbon.apimgt.impl.workflow.WorkflowExecutor abstract class.
- Need to override the following methods

    execute: contains the implementation of the workflow execution

    complete: contains the implementation of the workflow completion
- Compile the class and create the JAR file.
- Add the JAR to the APIM.
- Update the workflow configuration to use the new custom workflow.

Customizing a Workflow Extension

Configure Workflows in a Cluster

If you are working in a clustered API Manager setup with the Control plane, Gateway and Traffic Manager in separate servers, do the workflow configurations in the **Control plane.**

Configure Workflows in a Cluster

Custom Grant Types

WSO2

Writing Custom Grant Types

- OAuth 2.0 authorization servers provide support for four main grant types according to the OAuth 2.0 specification. They also allow you to add custom grant types and extend the existing ones.
- API Manager has already customized the Grant Type handlers for authorization_code , password , client_credentials and saml2-bearer grant types.
- If you require any additional functionality for these grant types, its advisable to extend the following grant handler implementations.

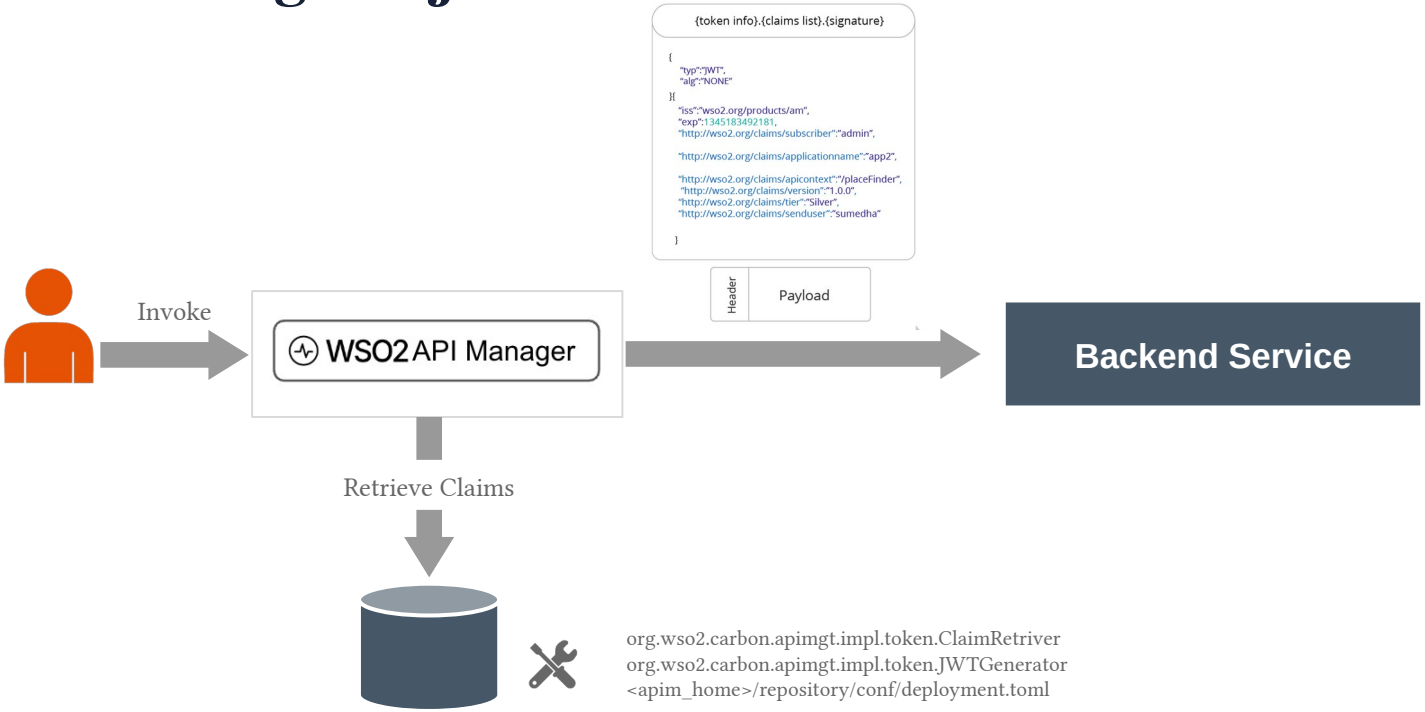
Grant Type	Existing Handler Class (which can be extended if required)
`authorization_code`	`org.wso2.carbon.apimgt.keymgt.handlers.ExtendedAuthorizationCodeGrantHandler`
`password`	`org.wso2.carbon.apimgt.keymgt.handlers.ExtendedPasswordGrantHandler`
`client_credentials`	`org.wso2.carbon.apimgt.keymgt.handlers.ExtendedClientCredentialsGrantHandler`
`urn:ietf:params:oauth:grant-type:saml2-bearer`	`org.wso2.carbon.apimgt.keymgt.handlers.ExtendedSAML2BearerGrantHandler`

Writing a Custom OAuth 2.0 Grant Type

Customizing JWT Generation

WSO2

Customizing the JWT Generation



Passing Enduser Attributes to the Backend using JWT

- JSON Web Token (JWT) is used to represent claims that are transferred between two parties such as the end user and the backend.

- A claim is an attribute of the user that is mapped to the underlying user store. It is encoded as a JavaScript Object Notation (JSON) object that is used as the payload of a JSON Web Signature (JWS) structure, or as the plain text of a JSON Web Encryption (JWE) structure. This enables claims to be digitally signed.

- When a request comes to the API Gateway, it retrieves the claims needed and builds a JWT token which is added to the header of the backend service request. The backend would get this and do the necessary validations.

Extending Scope Validation

WSO2

Scope Validation

- OAuth scopes allow you to have fine-grained access control to API resources based on the user roles.
- It allows you to define scopes per API and associate defined scopes with API resources. OAuth 2.0 bearer tokens are obtained for a set of requested scopes and is not allowed to access any API resources beyond the associated scopes.
- API Manager uses scopes as a way of defining permissions for a resource.
- If a resource is assigned a scope, then the token accessing the resource should be generated with that scope.
- By associating a scope with a role, you can control which users are permitted to have tokens under certain scopes.
- Validating the role of a requester does not make much sense in some scenarios.
- For instance, when the scope is used as a means of generating an access token and not for securing a resource (e.g., OpenID scope). In such situations, scope validation can be extended to skip role validation for certain scopes.

Extending Scope Validation

When scopes which cannot be associated to roles are requested, the token should be issued without validating the scope. In WSO2 API Manager, this is done by allowing the scope through configuration. Scopes that match the pattern are not validated by role and are available to anyone requesting it.

```
[apim.oauth_config]
allowed_scopes = ['^device_.*', 'openid', 'some_random_scope']
```

Extending Scope Validation

Extending API Lifecycle

WSO2

Extending the API Life Cycle

APIs created in WSO2 API Manager have their own life cycle consisting of the following:

- A set of life cycle states,
- Specific actions for each state transition
- A checklist of items before a state transition occurs.
- Configuration is done from Admin portal from APIM 4.2.0 onwards

Extending the API Life Cycle

Let’s try it out!

WSO2